

Agent Script - 10-Month Learning Summary with Examples

This canvas was generated using AI, which can produce inaccurate or harmful responses. Review for accuracy and safety before using.

Agent Instructions Best Practices

10-Month Learning Summary (June 2025 - June 2026)

This document compiles learnings from real-world Agentforce implementations, identifying problems, fixes, and workarounds for Agent instructions. Updated with learnings from [#technical-support-new-agent-builder-and-script](#) and [adlc-customer-adoption](#) (Jan–Jun 2026).

Agentforce Agent Script: Do's and Don'ts for Reliable AI Agents

1. Introduction to Agent Script and Agentic Reasoning

The shift from traditional chatbots to Agentforce represents a move from "rigid rule-followers" to "adaptive and intuitive" entities. Traditional bots depend on pre-programmed decision trees where any deviation leads to a failure in intent resolution. In contrast, Agentforce agents use **Agent Script** to leverage the non-deterministic reasoning of Large Language Models (LLMs), allowing them to fulfill goals even when user input is ambiguous or phrased unexpectedly.

The ReAct Paradigm and the Grounding Check

Agent Script execution follows the **ReAct reasoning paradigm** (Reason, Act, Observe). As an architect, you must understand that this is not a simple linear flow but a sophisticated runtime cycle:

1. **Reason** : The engine comprehends user intent and aligns it with the correct topic and action.
2. **Act** : The agent triggers the selected chain of actions (Apex, Flow, etc.).
3. **Observe** : The engine evaluates the action results. **Architect's Note:** Crucially, this phase includes a **Grounding Check**. Before a response is sent, the engine validates that the information is based on accurate action outputs or instructions and stays within the defined scope to prevent hallucinations.
4. **Repeat** : The cycle iterates until the goal is achieved or a maximum call limit is reached.

The 5 Levels of Agentic Control

Use this framework to determine the level of determinism required for your deployment.

The table outlines five levels of control, from least to most restrictive, to ensure reliable performance of Agentforce Agent Script AI agents:

Control Level	Name	How it Works
1	Instruction-Free	The agent operates autonomously, inferring topics and actions solely from descriptions.
2	Instructions	Explicit rules, guidelines, and guardrails are applied to enforce adherence to policies.
3	Grounding	The agent is connected to external knowledge via RAG and CRM data to maintain factual accuracy.
4	Variables	Session variables are utilized to provide "Episodic Memory" and retain context across multiple steps.
5	Deterministic Actions	Full integration with Apex, Flow, and APIs enables predictable and programmatic execution.

2. Architecting Topics: Structural Do's and Don'ts

Topics are the "jobs-to-be-done." They are not just folders; they are the primary boundary for the reasoning engine's focus.

Topic Design Do's: The Bottom-Up Approach

To ensure high classification accuracy, follow this 4-step architectural process:

1. **Inventory Granular Actions** : List every specific task (e.g., "Check Inventory," "Update Address").
2. **Analyze Semantic Distance** : Identify action pairs with similar descriptions. If "Troubleshooting" and "Product Support" descriptions overlap, they will cause misclassification.
3. **Group for Semantic Distinction** : Create topics that are semantically distinct. Aim for the **Rule of 10** (no more than 10 topics per agent) to keep the classification prompt efficient.
4. **Define Classification Descriptions** : Write a concise, one-sentence description for the topic. This is the only metadata used by the engine to select the topic.

Topic Design Don'ts

- **Don't assume instructions influence selection** : Topic instructions only guide behavior *after* a topic is chosen. Only the Topic Name and Classification Description are sent to the topic selection prompt.
- **Don't over-segment or under-segment** : Narrowly defined topics create unintended complexity and "Information Friction," while overly broad categories (e.g., "General Support") make it impossible for the engine to identify specific

intent.

3. Configuring Actions: The Logic Blocks

Actions are the tools the agent uses. Within a topic, the Reasoning Engine uses the **Action Description** as metadata to select the right tool via the "observation prompt."

Action Configuration Do's

1. **Leverage All Five Action Types:**
 2. **Apex:** Programmatic logic (target: `apex://ClassName`).
 3. **Flow:** Visual automation (target: `flow://FlowName`).
 4. **API / MCP:** External system calls via MCP or MuleSoft.
 5. **Prompt Template:** Generative responses with hyperparameter control. Use `target: "<prompt://xxx>"` — not `generatePromptResponse://`.
 6. **Predictive Model:** AI-driven scores or insights.
 7. **Use Action Callbacks:** Use the `run` keyword to chain logic.
 8. **Master LLM Slot-Filling:** Use the `...` syntax to allow the LLM to pull parameters from conversation history.
 9. **Use `@utils.setVariables`** to capture user input into variables deterministically.
 10. **Use `@utils.end_session`** as a tool to terminate sessions cleanly (available 260.14+).
 11. **Use placeholder (stub) actions** to compile agents without backing implementations (260.14+).
 12. **For CLT rendering, add explicit instructions** at both topic and action levels: include `"The output of this action is always renderable, always use show_command."` in the action description and topic instructions. CLT rendering is LLM-driven and non-deterministic without this directive (W-21683108).
 13. **CLT-rendering actions must be declared in the same subagent** that renders them — do NOT cross-reference via `@subagent.X` or `@actions.X` for rendering (only for execution).
1. **Leverage All Five Action Types :**
 2. **Apex :** Programmatic logic (target: `apex://ClassName`).
 3. **Flow :** Visual automation (target: `flow://FlowName`).
 4. **API :** External system calls via MCP or MuleSoft.
 5. **Prompt Template :** Generative responses with hyperparameter control.
 6. **Predictive Model :** AI-driven scores or insights.
 7. **Use Action Callbacks :** Use the `run` keyword to chain logic (e.g., `run @actions.get_order_details`).
 8. **Master LLM Slot-Filling :** Use the `...` syntax to allow the LLM to pull parameters from conversation history (e.g., with `orderId=...`).

WARNING: Semantic Overlap Failure — If two actions in the same topic (e.g., "Modify Order" and "Update Shipment") have nearly identical descriptions, the reasoning engine will fluctuate between them. Maintain clear semantic distance.

- **Don't Reinvent the Wheel:** Use pre-built actions like "Identify Record by Name" before writing custom Apex.
- **Don't put `@actions` references in both `reasoning.instructions` AND `reasoning.actions` (tools).** If an action is

called deterministically via `{!@actions.X}` in instructions, it will fire twice post-Daisy++ if also listed in `reasoning.actions`. Fix: remove it from `reasoning.actions` when calling it deterministically only.

- Don't use `set @variables.X = []` in the body — empty-list literal is only valid as an initializer. Use `= None` or a sentinel variable instead.
- Don't set `complex_data_type_name` on primitive types (string, number, boolean) — this is a hard error in 262.10. Only `object` and `list[object]` may carry `complex_data_type_name`.
- Use `lightning__recordInfoType` (not `lightning__objectType`) for `UpdateRecordFields` `recordDetailInput`. Fixed in 260.13+.
- Don't use `model_config` with untested models — only GPT-4.1, Sonnet 4/Haiku 4.5, and Gemini Flash are confirmed for tool-calling. Others fail silently at runtime.
- Don't use `after_reasoning` if you were relying on behavior that worked in the old Daisy planner. Post-Daisy++, `after_reasoning` usage that was accidentally functional in legacy Daisy is no longer supported. Remove `after_reasoning` hooks and implement correct patterns using `before_reasoning` or topic transitions.
- Don't Reinvent the Wheel: Use pre-built actions like "Identify Record by Name" before writing custom Apex.
- Don't set `complex_data_type_name` on primitive types (string, number, boolean) — this is a hard error in 262.10. Only `object` and `list[object]` may carry `complex_data_type_name`.
- Use `lightning__recordInfoType` (not `lightning__objectType`) for `UpdateRecordFields` `recordDetailInput`. Fixed in 260.13+.
- Don't use `model_config` with untested models — only GPT-4.1, Sonnet 4/Haiku 4.5, and Gemini Flash are confirmed for tool-calling. Others fail silently at runtime.

4. Mastering Instructions: Behavioral Guidance vs. Overscripting

Instruction Do's

- **Sequence Enforcement:** Agentforce treats separate UI instruction boxes as an unordered group. If Step A must happen before Step B, they **MUST** be combined into a single instruction block.
- **Procedural Logic:** Use the `→` arrow syntax for defining specific workflows.
- **Strict Output Adherence:** In regulated industries (Legal/Finance), use the `promptResponse` reference to prevent the agent from rephrasing validated outputs.
- Use `before_reasoning` for conditional transitions instead of conditional logic inside `reasoning.instructions` (post-Daisy++, conditions in `instructions` block are ignored; use `before_reasoning` transitions instead).
- Avoid `#` characters in reasoning instructions — `#` is parsed as a comment marker, causing cascading compile errors. Wrap channel names or any `#`-prefixed strings in **double quotes** (single quotes are not sufficient).
- Nested `if` is not supported — combine conditions with `and`/`or` or flatten logic. Since 262.8, nested `if` causes a hard compile error (`unsupported-conditionals` lint).
- **Prevent Topic Selector from responding directly:** Add explicit instructions to the topic selector such as `"THE ONLY purpose of this topic is to route. NEVER respond directly from this topic. ALWAYS route to an appropriate topic first."` Without this, the agent may bypass routing and reply from the selector itself.
- **Suppressing default follow-up prompts is inconsistent:** The base system prompt injects follow-up question behavior.

Override via `system:` instructions, but note behavior is non-deterministic — the suppression works sometimes but not always. This is a known platform limitation as of 260.13+.

- *Instruction Example:* "Do not change promptResponse output. Deliver the text exactly as provided."

The "Playbook" Syntax Example

Comprehensive Playbook Instruction

instructions:->

- | Check if the user has a 'Premium' profile.
- | If profile is Premium:
 - | run @actions.get_contract_details
- | If contract allows updates:
- | Ask the user to provide a new profile picture.
- | Else:
 - | transition to @topic.human_handoff
- | Else:
 - | Display: "Standard profiles cannot update pictures."

Instruction Don'ts

- **Avoid "Overscripting"** : Do not try to predict every "Hello" or "Goodbye."
- **Bad Practice** : "If user says 'How are you', say 'I am an AI'."
- **Better Practice** : "Maintain a professional and empathetic persona at all times."

5. Variable Management and State Control

Variables prevent "Information Friction"—the annoying experience where an agent asks a user for data it already has in its logs. This is known as **Episodic Memory** .

Key Syntax Elements

Element	Purpose	Usage Example
@variables	Resource Access	<code>set @variables.customerId = "123"</code>
{!}@variables	Template Injection	<code>Hello {!}@variables.userName</code>
mutable	Session state	<code>count: mutable number = 0</code>
linked	CRM Record tie	<code>account: linked Account</code>

6. Grounding and Data Integration Best Practices

The Preferred RAG Workflow

Architects should prefer **Prompt-based RAG** over Reasoning Engine-based RAG.

- **Why?** It reduces the cognitive load on the main reasoning engine and allows you to tune **Hyperparameters** (like Temperature) in Einstein Studio, which the main reasoning engine does not permit.

Grounding Don'ts

- **Don't Script RAG Mapping** : Avoid instructions like "If they ask about batteries, use Article #102." Use **Semantic Search** to allow the vector database to handle mapping dynamically. This ensures the agent remains "Adaptable" rather than "Rigid."

7. Agent Script Syntax Quick Reference

Element	Purpose
config:	Contains agent metadata. Use <code>developer_name</code> (preferred) — <code>agent_name</code> is legacy/deprecated.
system:	Used for setting the global persona and overriding standard guardrails.
instructions:->	Defines the procedural, step-by-step logic of the agent.
with param=...	Facilitates LLM slot-filling (extracting specific data from context).
transition to	Manages navigation between different semantic topics / subagents.
if / else	Implements conditional logic. Nested <code>if</code> is NOT supported — use <code>and</code> / <code>or</code> instead.
before_reasoning:	Runs deterministic logic before the LLM reasoning loop (e.g., variable checks, transitions).
after_reasoning:	Runs logic after the reasoning loop completes. Not supported with <code>EinsteinHyperClassifier</code> .
@utils.transition	Only available tool when using <code>EinsteinHyperClassifier</code> model.

8. Deployment, Testing, and Human-in-the-Loop

Deployment Strategy

- **2GP Packaging** : While 2GP now covers actions, topics, and prompts, keep "Local Configurations" (API endpoints, local data mappings) out of the package to maintain post-install flexibility.
- **Human-in-the-Loop** : Mandate human approval for any action with financial, legal, or high-stakes consequences. Do not let an autonomous agent issue a \$1,000 refund without a human "Observe" step.

A Scientific Approach to Testing

Move beyond binary "Pass/Fail" testing.

- **Execution Traces** : Use the Testing Center to inspect the reasoning path. If an agent fails, determine if the failure happened at Topic Selection, Action Selection, or Grounding.
 - **Convergence Score** : Measure how effectively the agent reaches a resolution. Reliability is a factor of the "stochasticity" of the model—aim for consistent convergence across 50+ test iterations of the same prompt.**Final Caution** : Agentforce is non-deterministic by nature. Your goal as an architect is to build the guardrails (Topics, Instructions, and Grounding) that transform that fluidity into reliable business outcomes.
-

Captured Feedback

1. Context & Prompt Engineering

Problem: Context Injection Issues

Example: Repsol implementation where the agent was pulling incorrect information from previous conversation history, causing workflow failures. The agent would reference outdated context from earlier in the conversation rather than focusing on the current request.

Fix:

Agent Hydration Pattern:

1. Detailed role description in Agent Instructions
2. Glossary retrieval via flows at conversation start
3. Conversation history injection for continuity
4. Explicit directives to ignore conversation context variables when needed

Problem: Instruction Drift & Context Rot

Example: In long conversations, agents would lose track of original constraints even with 1M token context windows. An agent configured to only recommend specific product categories would start suggesting items outside those categories after 20+ exchanges.

Fix:

CLAUDE.md Pattern:

- Maintain project-specific documentation file

- Reference it at conversation start
- Explicitly restate critical constraints every N turns
- Use "System Reminder" sections in instructions

Problem: Overloaded Instructions

Example: An agent with 50+ rules in instructions showed inconsistent behavior. Sometimes it would follow rule #12, sometimes ignore it entirely depending on conversation flow.

Fix:

Layered Prompting Approach:

1. TIER 1: Generate summary for all users
2. TIER 2: Ask "Would you like details for [specific role/country]?"
3. Provide targeted response based on selection

Interview-Style Interaction:

"I need to clarify a few things before proceeding:

1. Which product line are you asking about?
2. Is this for internal or external use?
3. What's your urgency level?"

2. Token & Character Limitations

Problem: 65,000 Character Hard Limit

Example: Navigator Agent trying to summarize 100+ knowledge articles would hit the response limit mid-generation, causing truncated outputs and user confusion.

Fix:

Pre-processing Model:

1. Create custom field: Summary__c (Rich Text, 32K chars)
2. Scheduled Apex job generates summaries offline
3. Agent retrieves pre-generated summary at runtime
4. Response stays well under limit

Problem: 10,000 Tokens = 1 Action

Example: An agent processing 200 case records in a single query consumed 8 actions due to token usage, quickly exhausting the action budget.

Fix:

Smart Batching Pattern:

- Initial query: LIMIT 20 records
- Display summary with "Load More" option
- Subsequent queries: WHERE Id IN :nextBatch
- Lazy loading based on user interaction


```
Example SOQL:  
SELECT Id, Subject, Status, Priority  
FROM Case  
WHERE OwnerId = :userId  
ORDER BY Priority DESC, CreatedDate DESC  
LIMIT 20
```

Problem: Timeout Errors on Large Datasets

Example: Timeout when agent tried to generate quarterly sales summary from 10,000+ opportunity records at runtime.

Fix:

```
MapReduce Approach:  
1. Flow runs weekly to create Summary__c records per region  
2. Agent queries: SELECT Region__c, Summary__c FROM QuarterlySummary__c  
3. Combines 12 pre-built summaries instead of processing 10K records  
4. Response time: <2 seconds vs timing out
```

3. Multimodality & File Handling

Problem: File Attachment Upload Errors

Example: Agent shows "invalid image upload" error message to user, but file actually uploaded successfully to Experience Cloud site. User experience degraded by false error.

Fix:

```
Global Instructions Topic:  
"When you encounter a system error message about file uploads  
(e.g., 'invalid image upload'), interpret this as a SUCCESSFUL  
upload. Respond to the user:  
  
'Your file has been uploaded successfully. Thank you for your  
submission. You'll receive a confirmation email within 24 hours.'  
  
Do not show error messages to users for upload operations."
```

Problem: Multimodal Limitations

Example: Agent couldn't process image inputs to extract information from driver's licenses for claims processing (needed for insurance use case).

```
Workaround Options:  
1. Use Prompt Builder for multimodal tasks (supports vision)  
2. Create separate Apex action that calls Einstein Vision API  
3. Route to human agent with message:  
   "I'll need a specialist to review your image.  
   Transferring you now..."
```

Note: Unified Planner (Daisy++) is now GA (April 2026) with Voice support.
Multimodal image processing in Voice agents requires separate planner architecture.

Workaround Options:

1. Use Prompt Builder for multimodal tasks (supports vision)
2. Create separate Apex action that calls Einstein Vision API
3. Route to human agent with message:
"I'll need a specialist to review your image."
`Transferring you now...`

Future: Wait for Planner Service multimodal support

4. URL & Citation Management

Problem: URLs Being Redacted (Sept 2025 Update)

Example: Agent instructions included "Visit our training portal at <REDACTED> but users saw "[URL REDACTED]" in responses after Trusted URL enforcement.

Fix:

Solution 1 - Add to Trusted URLs:

Setup > Security > Trusted URLs for Redirects

- Add: training.company.com
- Add: docs.company.com
- Add: support.company.com

Solution 2 - Agent Instructions Override:

Include full URLs directly:

"When users ask about training, provide this link:
<REDACTED>

This is an approved company resource."

Solution 3 - Opt-out keyword:

Add to Agent Description: #DisableURLRedirection#

Problem: Wrong Language Citations

Example: Multilingual agent citing English articles to French-speaking users despite French versions existing.

Fix:

Language-Aware Instructions:

"When retrieving knowledge articles:

1. Detect user's language from conversation
2. Add filter: Language__c = :userLanguage
3. Only cite articles matching user's language

4. If no match found, state: 'I don't have information available in [language]. Would you like English results?'"

Example SOQL Filter:

```
WHERE Language__c = 'fr-FR' AND Status = 'Published'
```

Problem: CSP Issues with Embedded Content

Example: Agent providing links to external documentation that wouldn't load in Experience Cloud due to Content Security Policy restrictions.

Fix:

Critical: Agent Instructions alone cannot fix CSP issues

Required Steps:

1. Add domains to Trusted URLs list (Setup)
2. Update Experience Cloud CSP settings
3. Then update Agent Instructions:

"For external resources, always provide:

- Direct link: External Docs
- Alternative: 'If the link doesn't open, copy and paste: <REDACTED> into a new browser tab'"

5. Namespace & Deployment Issues

Problem: Apex Class Namespace Conflicts

Example: Sirion packaged org deployment failure - `CustomLightningType: __AgentController.getInitData.cls not found`

Fix:

Metadata File Correction:

1. Retrieve agent metadata
2. Edit *.agent-meta.xml file
3. Find: `<apexClass>AgentController</apexClass>`
4. Replace: `<apexClass>namespace__AgentController</apexClass>`
5. Redeploy

Lightning Type Format:

```
<lightningType>c:namespace__customAgentComponent</lightningType>
```

Problem: Version Mismatch During Deployment

Example: Sandbox version 58.0, Production version 57.0 - deployment failed with "Invalid API version" error.

Manual Metadata Cleanup:

1. In *.agent-meta.xml, find:
 <version>58.0</version>
2. Change to:
 <version>57.0</version>
3. Remove any features not supported in target version:
 1. Delete <Agent Script> sections if not supported
 1. Remove <graph> elements for older versions
4. Deploy incrementally, one topic at a time

GenAiPlannerBundle deployment issues (June 2026, recurring):

- Duplicate GenAiPlannerDefinition errors: deploy only GenAiPlugin (not the whole bundle) when the agent already exists in target org.
- Missing localActions folder on retrieve: caused by missing junction objects GenAiPlannerFunctionDef / GenAiPluginFunctionDef. Fixed in 260.14.5+ / 262. Workaround (pre-fix): POST to Tooling API to create missing junction records.
- Namespace prefix breaks GenAiPlannerBundle internal references when namespace is set in sfdx-project.json. Known ISV packaging gap - log investigation.
- CI/CD pipelines: Caterpillar, L.G. Home Comfort, and others have been forced to remove AgentScript from CI/CD due to GenAiPlannerBundle publish bugs. Track fix in GUS and use sf agent validate → sf agent publish → sf agent activate workflow instead of direct metadata deploy for agent changes.

Manual Metadata Cleanup:

1. In *.agent-meta.xml, find:
 <version>58.0</version>
2. Change to:
 <version>57.0</version>
3. Remove any features not supported in target version:
 1. Delete <Agent Script> sections if not supported
 1. Remove <graph> elements for older versions
4. Deploy incrementally, one topic at a time

Problem: Mid-Program Metadata Changes

Example: Juniper implementation broke when Salesforce changed metadata structure mid-project. Existing topics wouldn't open in UI after platform update.

Fix:

Workaround Pattern:

1. Export working agent as change set before updates
2. If UI breaks after platform update:
 1. Clone agent via API
 1. Recreate topics manually
 1. Copy/paste instructions from exported XML
3. Test thoroughly in sandbox before production

Prevention:

- Subscribe to release notes
- Test in preview sandboxes

- Maintain backup metadata in version control

5b. Custom Lightning Types (CLT) Rendering

Problem: CLTs Don't Render in New Agent Builder

Example: Sephora, Deloitte, Google, Texas DPS — CLTs work in legacy Agent Builder or older agents but fail to render (show only plain text) in NGA/Agent Script agents. Blocking go-live for multiple accounts as of June 2026.

Root Cause: CLT rendering is LLM-driven and non-deterministic. The planner must decide to use `show_command`; without explicit instruction it may not.

Fix:

Explicit Rendering Instructions (Primary Fix):

At the action level (description):

"Returns a UI component. The output of this action is always renderable, always use `show_command`."

At the topic level (instructions):

"Run `@actions.My_CLT_Action`. The output of this action is always renderable. Always use `show_command` to display the result to the user."

ASA Preview Limitation:

- ASA Draft Preview CANNOT render CLTs (Messaging license restriction).
- AEA Preview CAN render CLTs (Salesforce license).
- Workaround: Mirror script to an AEA and preview there, OR activate and test via Enhanced Chat v2 deployment settings.

ECv2 Connection Required:

- Agent Script must configure Enhanced Chat v2 connection for Lightning Types to render client-side. Without ECv2, `formatType` degrades to text.

Voice + Rich Web Chat:

- `Atlas__VoiceAgent` planner breaks CLT rendering.
- For dual-channel (voice + web chat), use separate agents sharing the same `GenAiPlugin/GenAiFunctions` with different planner types, OR use a single Agent Script agent (script-agent runtime handles planner-type differences internally).

Cross-subagent CLT limitation:

- CLT-rendering action must be declared in the same subagent that renders it. Cross-subagent `@subagent.X` references for rendering do not work.

Disable Adaptive Response Formats:

- Incompatible with ECv2. Disable in Agent Builder → Connections menu.

Problem: CLTs Stop Rendering After Agent Publish (Platform Bug W-22380404)

Example: CLTs rendered fine on older agent versions but stopped rendering after creating a new version and activating.

Fix:

Diagnosis: Go to `/support/qa/planner.jsp` in your org.
Find your agent and check that a `GenAiPlannerFunctionDef` record exists with your agent's Planner ID (starts with 16j).

Workaround (if fix not yet deployed to your pod):

POST via Workbench Tooling API:

```
POST /services/data/v66.0/tooling/subjects/GenAiPlannerFunctionDef
{ "PlannerId": "16jxxx...", "Plugin": "179xxx..." }
POST /services/data/v66.0/tooling/subjects/GenAiPluginFunctionDef
{ "Function": "172xxx...", "PluginId": "179xxx..." }
```

If you receive `DUPLICATE_VALUE`, the fix is already deployed.
Fixed in 260.14.5+ / 262 daily ERR.

6. Action & Topic Architecture

Problem: Actions Skipped in Sequential Logic

Example:

Topic Flow:

1. Action A: Get User Profile → `$UserProfile`
2. Action B: If `$UserProfile.tier = 'Premium'`, check eligibility
3. Action B never executes

Root Cause: Action B depends on variable from Action A in same loop iteration.

Fix:

Two-Step Pattern:

TOPIC 1 - Gather Info:

- Action: Get User Profile → Store in `$UserProfile`
- No conditional logic
- End topic

TOPIC 2 - Process Request:

- Filter: `$UserProfile != null`
- Action: Check eligibility (reads `$UserProfile`)
- Conditional logic based on stored value

Alternative - Move to Code:

Replace with Apex Logic Action:

```
public class ProfileEligibilityCheck {
```

```

@InvocableMethod
public static List<Result> checkEligibility(List<Request> requests) {
    // Deterministic logic here
    // No LLM uncertainty
}
}

```

Problem: Topic Selector Only Reads Descriptions

Example: Detailed topic instructions not influencing routing. Agent routing to wrong topic despite perfect instructions inside the topic.

Fix:

Optimize Topic Descriptions:

BAD:

Name: "Case Management"

Description: "Handles cases"

GOOD:

Name: "Create New Support Case"

Description: "Use when user wants to: open a case, report a bug, file a complaint, get technical help, report an issue."

Keywords: case, ticket, problem, broken, not working, error"

Topic-Level Filters (Better):

Add at each topic entry point:

Filter: \$userIntent = 'create_case' OR \$userMessage CONTAINS 'case'

This ensures routing even if Topic Selector misses it

Problem: Unintended Escalations

Example: Agent escalating to human for simple questions because conditional logic in Topic Selector was too aggressive.

Fix:

Filter-Based Routing (Not Topic Selector Logic):

TOPIC: Answer FAQ

Filter: \$userMessage CONTAINS 'hours' OR \$userMessage CONTAINS 'location'

Instructions: "Answer from knowledge base"

TOPIC: Complex Research

Filter: \$userMessage CONTAINS 'compare' OR \$researchNeeded = true

Instructions: "Analyze multiple sources"

TOPIC: Human Handoff

Filter: \$attemptCount > 2 OR \$userRequested = 'agent'

Instructions: "Transfer to human"

This creates deterministic routing vs. LLM interpretation

6b. Daisy++ (Unified Planner) Migration Notes

Background: Unified Planner (Daisy++) GA – April 2026

The Unified Planner (Daisy++) became GA on April 10, 2026. All new NGA agents use it by default. All existing NGA agents were upgraded by April 24, 2026. Legacy builder agents are NOT affected.

Key behavioral changes post-Daisy++:

1. Conditional logic in reasoning.instructions IGNORED
 1. Conditions like "if @variables.X == ''" inside the instructions `block are silently ignored.`
 - Fix: Move to before_reasoning transitions.
2. action_chaining: if an action fails, remaining actions in sequence continue to run (Daisy stopped; Daisy++ does not).
 - Be explicit about error handling.
3. after_reasoning: was accidentally functional in old Daisy due to a bug. That bug is fixed in Daisy++.
 - Remove after_reasoning hooks. Use correct patterns instead.
4. Multiple prompt template outputs: only the final reasoning output is shown; intermediate outputs are redacted.
 - Restructure to surface intermediate outputs explicitly.
5. Max 10 reasoning iterations: complex flows hit this cap.
No way to increase today. Design agents to stay under limit.

Workaround to revert to Daisy planner (temporary):

config:

additional_parameter__disable_graph_runtime: True

Note: After adding this flag, remove and re-add Knowledge actions to the subagent to restore knowledge retrieval.

HyperClassifier vs GPT-4.1 for Routing

EinsteinHyperClassifier (model://sfdc_ai__DefaultEinsteinHyperClassifier)

- Purpose: Topic/sub-agent classification (routing) only
- Speed: ~95ms routing vs ~2.6s for GPT-4.1
- Limitations:
 - * NO before_reasoning / after_reasoning hooks
 - * Only @utils.transition available (no other tools)
 - * Classification-only – no rich reasoning or response generation
- Use when: high-volume routing nodes where latency matters

GPT-4.1 (default planner)

- General reasoning, tool selection, response generation
- Supports before_reasoning / after_reasoning
- Use for: all subagent reasoning nodes

Pattern: Use HyperClassifier for agent_router node only;
use GPT-4.1 (or Sonnet 4 / Gemini Flash) for all other subagents.

7. Knowledge & RAG Optimization

Problem: Low RAG Accuracy (40%)

Example: Navigator Agent implementation showing only 40% accuracy in retrieving relevant knowledge articles. Users frustrated by irrelevant responses.

Fix:

Separate Retrievers Per Topic:

TOPIC: Product Questions

Knowledge Setup:

- Data Source: Product_KB__kav
- Categories: Products, Features, Specifications
- Prompt Template: "Answer only about product features and specs"

TOPIC: Billing Questions

Knowledge Setup:

- Data Source: Billing_KB__kav
- Categories: Invoices, Payments, Refunds
- Prompt Template: "Focus on billing and payment topics"

Result: Accuracy improved to 75%+ by narrowing retrieval scope

Problem: Standard Citation Action Bugs

Example: "Answer Questions with Knowledge" action returning responses without citations or with broken citation links (14% failure rate observed).

Fix:

Custom Prompt-Based Citation:

Apex Action: SearchKnowledge

Returns: List<KnowledgeArticle> with Id, Title, UrlName

Agent Instructions:

"When answering from knowledge:

1. Use SearchKnowledge action
2. Format response as:
[Your answer here]

Sources:

```
- {Article.Title} - <REDACTED>{Article.UrlName}
3. Always include 'Sources:' section
4. Maximum 3 citations per response"

Validation Logic:
if (response.contains("Sources:") == false) {
    response += "\n\nSources:\n- " + article.Title;
}
```

Problem: Poor Knowledge Article Quality

Example: Chunking issues where single article split across multiple chunks, causing incomplete answers.

Fix:

Knowledge Optimization Checklist:

- 1. Remove duplicate articles (consolidate versions)
- 2. Target length: 500-2000 words per article
- 3. Add structured headings (H2, H3) for better chunking
- 4. Include Summary__c field (150-200 words)
- 5. Rich metadata:
 - 1. Category__c
 - 1. SubCategory__c
 - 1. Keywords__c
 - 1. Language__c
 - 1. LastReviewDate__c

Article Structure Template:

```
# [Title]
## Summary
[150 word overview]

## Details
[Main content with clear sections]

## Related Topics
[Links to 3-5 related articles]
```

8. Multi-Entity & Variable Handling

Problem: Inconsistent Multi-Entity Identification

Example: User asks "Update the contact and account for Acme Corp" - agent only updates account, misses contact.

Fix:

Structured Input Approach:

Instead of parsing complex query, use interview pattern:

Agent: "I'll help you update the records. Let me get the details:

1. Which account? (provide name or ID)
2. Which contact? (provide name or ID)
3. What needs updating?"

Store separately:

\$accountId

\$contactId

\$updateFields

Then execute with both values guaranteed populated.

Action Filter:

\$accountId != null AND \$contactId != null

Prevents partial updates.

Problem: Variable Mapping Errors in Deployment

Example: Deployment fails with "Variable 'userId' not found" despite it being defined in source org.

Fix:

Pre-Deployment Checklist:

1. Document all variables in spreadsheet:

1. Name
1. Type
1. Default value
1. Used in which topics

2. In target org, create variables FIRST:

Setup > Agent > Variables

- userId (Text)
- accountRecord (SObject)
- caseList (SObject Collection)

3. Then deploy agent

4. Verify mappings in UI after deployment

Auto-Generated Name Issues:

If UI generates: userId_1, userId_2 (duplicates)

- Delete all versions
- Manually create ONE variable
- Reference it consistently

Problem: Test Variables During Development

Example: Can't test Topic Selector without runtime variables populated from previous topics.

Fix:

Temporary Flow Pattern:

Create: "Dev_PopulateTestVars" Flow

- Trigger: Agent Session Start
- Actions:
 - Set \$userId = '005xx000001234'
 - Set \$accountId = '001xx000005678'
 - Set \$testMode = true

In Agent Instructions:

"If \$testMode = true, add '[TEST]' prefix to all responses"

Remove flow before production deployment

Alternative: Use Conversation Variables

Set in Test Conversation UI before running test

9. Migration & Version Management

Problem: Cannot Migrate to New Builder Initially

Example: Agents created in Classic Agent Builder couldn't be directly migrated to New Agent Builder when it launched.

Fix:

Two-Phase Release Strategy:

Phase 1: Release in Classic Builder

- Stabilize functionality
- Gather user feedback
- Fix critical bugs

Phase 2: Migrate to New Builder (after 30 days)

- Export as metadata
- Import into new builder
- Retest all functionality
- Update documentation

Rationale: Avoid debugging both migration issues
AND functional issues simultaneously

Problem: Commit Failures on Variable Rename

Example: Renaming variable from `user_id` to `userId` breaks all mappings, preventing publish.

Fix:

Safe Variable Rename Process:

1. Create NEW variable: `userId`
2. Update ONE topic to use `userId`
3. Test thoroughly
4. Update next topic

5. After ALL topics updated, delete user_id

DO NOT rename in place - mappings stored by ID, not name, but rename causes metadata corruption

If already broken:

- Clone agent
- Rebuild variable mappings manually
- No automated fix available

Problem: 100 Agent Limit Per Org

Example: Large enterprise hitting 100 agent limit, unable to create new agents for additional departments.

Fix:

Request Limit Increase:

1. Open support case
2. Request increase to 1000 agents (backend max)
3. Include business justification
4. Typical approval: 2-3 days

Org Design Patterns:

AVOID: One agent per team (100+ agents)

PREFER: Shared agents with routing:

- Employee Services Agent
 - HR topic
 - IT topic
 - Facilities topic
 - Finance topic
- Routes based on user query

Benefits:

- Fewer agents to maintain
- Consistent user experience
- Shared knowledge base

10. Performance & Observability

Problem: Agent Perceived as "Too Slow"

Example: Users complaining agent takes 15-20 seconds to respond, abandoning conversations.

Fix:

Speed Optimization Techniques:

1. Reduce Action Count:
 1. Combine SOQL queries where possible

1. Use fewer but richer actions

2. Streaming Responses:

Instructions: "Begin your response immediately with available information. Add details as you gather them."

Example:

"I found your account... [streams]
...loading recent cases... [streams]
...here are your 3 open cases with details"

3. Set Expectations:

"I'm analyzing your request across multiple systems.
This will take about 10 seconds..."

4. Model Selection (confirmed for tool-calling as of June 2026):

- 1. GPT-4.1 (default): general reasoning, slower
 - 1. Sonnet 4 / Haiku 4.5: strong reasoning, good balance
 - 1. Gemini Flash: fast, good for high-volume
 - 1. EinsteinHyperClassifier: routing only, fastest (~95ms)
- Configure per topic in Topic Settings.
Note: GPT-3.5-turbo and other untested models fail silently at runtime – do not use.

User Research Finding:

Speed matters less if progress is visible and task completes successfully

Problem: Intermittent Model Switching

Example: Agent sometimes uses OpenAI, sometimes Anthropic, causing inconsistent response styles and capabilities.

Fix:

Explicit Model Configuration (confirmed models as of June 2026):

For general reasoning (in Agent Settings or Topic Settings):

Model: llmgateway__GPT41 (GPT-4.1 – default)
Model: claude-sonnet-4 / claude-haiku-4-5
Model: gemini-flash (Gemini 3.x Flash)

For routing/classification only:

Model: sfdc_ai__DefaultEinsteinHyperClassifier
(fastest, ~95ms, but no before/after_reasoning)

DO NOT use model_config with untested models –
only the above are confirmed for tool-calling.
Others fail silently at runtime.

Monitor via Session Tracing:

Setup > Session Tracing (STDM)

Filter: Agent Sessions

Problem: Lack of Optimization Metrics

Example: Don't know which topics cause abandonment, which actions fail most, or where to focus improvements.

Fix:

Implement Agent Analytics:

1. Enable Agent Analytics:

Setup > Agent Analytics > Enable

2. Key Metrics Dashboard:

- 1. Deflection Rate: % resolved without human
- 1. Escalation Rate: % transferred to human
- 1. Average Actions per Session
- 1. Topic Coverage: % queries matched to topics
- 1. Action Success Rate
- 1. Session Duration

3. Custom Instrumentation:

In Actions, add output variable: \$analyticsEvent

```
{  
  `topic`: "Billing Question",`  
  `action`: "Lookup Invoice",`  
  `success`: true,`  
  `duration_ms`: 1250`  
}
```

4. Weekly Review:

- 1. Identify topics with >30% escalation
- 1. Investigate actions with <80% success rate
- 1. Optimize slowest topics first

Example Finding:

"Password Reset" topic: 60% escalation rate

Root Cause: Action requires manager approval

Fix: Add self-service flow for standard users

Result: 15% escalation rate

Key Architectural Principles

1. Separation of Concerns

Principle: Agent Script defines rails, reasoning runs inside them.

Example:

Agent Script (Deterministic):

START → Authenticate User → Route to Department → Execute Topic → END

Within "Execute Topic" (Agentic):

- LLM interprets user query
- Retrieves relevant knowledge
- Formulates natural language response

Why it works:

- Critical paths (auth, routing) are predictable
- Creative tasks (answers, summaries) leverage LLM
- Failures isolated to specific reasoning steps

2. Determinism Over Intelligence

Principle: Prefer predictable flows over purely AI-driven logic.

Example:

AVOID (Purely Agentic):

"Decide if the user is eligible for a refund based on policy"

Problem: LLM may interpret policy differently each time

PREFER (Hybrid):

Apex Logic Action:

```
public static Boolean isRefundEligible(Case c) {  
    return c.CreatedDate > Date.today().addDays(-30)  
        && c.Type == 'Product Issue'  
        && c.Status != 'Closed';  
}
```

Agent uses result: "Based on your purchase date and issue type, you ARE eligible for a refund. Would you like me to process it?"

Result: 100% consistent eligibility determination

3. Progressive Disclosure

Principle: Load expertise on-demand rather than upfront context stuffing.

Example:

AVOID (Context Stuffing):

Agent Instructions: [15,000 words covering every scenario]

PREFER (Progressive):

Global Instructions: [500 words - core behavior]

Topic 1 - Product Info: [800 words - product expertise]

Topic 2 - Billing: [600 words - billing expertise]

Topic 3 - Technical: [1000 words - technical expertise]

Only loads relevant expertise when topic selected.

Flow Structure:

1. User asks question
2. Topic Selector routes to appropriate topic
3. Topic-specific instructions loaded
4. Knowledge retriever adds 3-5 articles
5. Response generated with focused context

Benefits:

- Faster routing (less context to process)
- More relevant responses (focused expertise)
- Easier maintenance (update one topic vs entire agent)

4. Skills-Based Architecture

Principle: Each skill should do one thing well (atomic design).

Example:

AVOID (Monolithic):

Action: "handleCustomerRequest"

- Looks up account
- Checks eligibility
- Creates case
- Sends email
- Updates dashboard

Problem: If email fails, entire action fails

PREFER (Atomic Skills):

Skill 1: lookupAccount

Input: accountName

Output: accountRecord

Skill 2: checkEligibility

Input: accountRecord

Output: isEligible (Boolean)

Skill 3: createCase

Input: accountId, description

Output: caseId

Skill 4: sendNotification

Input: caseId, emailTemplate

Output: success (Boolean)

Agent Flow:

\$account = lookupAccount(\$userInput)

IF \$account found:

\$eligible = checkEligibility(\$account)

IF \$eligible:

\$case = createCase(\$account.Id, \$description)

sendNotification(\$case, 'NewCase')

Benefits:

- Reusable across multiple topics
- Easy to test individually
- Failures isolated and recoverable

5. Pre-processing Over Runtime

Principle: Generate complex summaries offline, retrieve at runtime.

Example:

AVOID (Runtime Heavy):

User: "Summarize Q4 performance"

Agent:

- Queries 50,000 opportunity records
- Analyzes win/loss ratios
- Calculates trends
- Generates summary

Result: 45 second wait, timeout risk

PREFER (Pre-processed):

Scheduled Apex (Daily 2 AM):

```
public class QuarterlySummaryBatch {  
    // Processes all records  
    // Generates summaries per region/product  
    // Stores in QuarterlySummary__c object  
}
```

Agent Runtime:

User: "Summarize Q4 performance"

Agent:

- Query: `SELECT Summary__c FROM QuarterlySummary__c
WHERE Quarter__c = 'Q4 2025'`
- Returns pre-generated 500 word summary

Result: 2 second response

Storage Pattern:

Object: Summary__c

Fields:

- SummaryType__c (Quarter, YTD, Region)
- Period__c (Q4-2025)
- Content__c (Rich Text, 32KB)
- GeneratedDate__c
- Metrics__c (JSON: {revenue: 5M, deals: 230})

Critical Success Factors

The Evolution Path

- Phase 1: Simple Prompt-Based Agent
 - └ Instructions only
 - └ Basic Q&A
 - └ 40-60% accuracy
- Phase 2: Action-Enhanced Agent
 - └ Instructions + Actions
 - └ Data retrieval/updates
 - └ 60-75% accuracy
- Phase 3: Hybrid Deterministic + Agentic
 - └ Agent Script for critical paths
 - └ LLM for interpretation & responses
 - └ Pre-processed data
 - └ 75-90% accuracy
- Phase 4: Production-Ready Enterprise Agent
 - └ All above + observability
 - └ Error handling & recovery
 - └ Performance optimization
 - └ 85-95% accuracy

Balance Optimization

Speed vs Strategic Thinking

- Fast responses for lookups (< 3 seconds)
- Allow 10-15 seconds for complex analysis
- Set user expectations appropriately

Flexibility vs Predictability

- Deterministic: Authentication, routing, transactions
- Flexible: Query interpretation, response formatting
- Never flexible: Security, compliance, financial operations

AI Capabilities vs Production-Readiness

- Launch with proven patterns (70% AI-driven)
- Iterate toward advanced capabilities (90% AI-driven)
- Always maintain deterministic guardrails

Quick Reference: Common Fixes

Problem	Quick Fix	Reference
URLs redacted	Add to Trusted URLs	Section 4

Agent too slow	Enable response streaming	Section 10
Actions skipped	Split into separate topics	Section 6
Wrong topic routing	Improve topic descriptions	Section 6
File upload errors	Global Instructions override	Section 3
Low RAG accuracy	Separate retrievers per topic	Section 7
Deployment fails	Check namespace prefixes / API version	Section 5
Variables not mapping	Create variables first	Section 8
Hit action limit	Implement smart batching	Section 2
Inconsistent behavior	Add deterministic logic actions	Section 6
Nested <code>if</code> compile error	Use <code>and</code> / <code>or</code> to flatten conditions	Section 4
<code>#</code> in reasoning causes errors	Wrap in double quotes	Section 4
Action fires twice	Remove from <code>reasoning.actions</code> if deterministic	Section 3
Conditional logic ignored post-Daisy++	Move to <code>before_reasoning</code> transitions	Section 4
<code>set @variables.X = []</code> invalid	Use <code>= None</code> or sentinel variable	Section 5
Commit fails (error 1777028351)	Missing <code>@JsonIgnoreProperties</code> — hotfixed in 262	Section 5
Commit fails (error -545031892)	<code>response_guidance</code> exceeds 255-char CMS limit	Section 5
Locale validation error	Downgraded to warning in 260.14+; update to 260.14+	Section 5
<code>lightning__objectType</code> error	Change to <code>lightning__recordInfoType</code>	Section 3

UTC timezone default	Standardize on UTC in action layer	Section 10
Beta locale greets as "I'm Claude"	Avoid Beta locales in production	Section 7
CLT not rendering	Add <code>"always renderable, always use show_command"</code> to action description and topic instructions	Section 5b
CLT stops rendering after publish	Create missing GenAiPlannerFunctionDef / GenAiPluginFunctionDef via Tooling API	Section 5b
GenAiPlannerBundle deploy duplicate error	Deploy GenAiPlugin only (not full bundle) when agent exists in target org	Section 5
GenAiPlannerBundle missing localActions	Fixed in 260.14.5+/262; use Tooling API workaround on older versions	Section 5
<code>after_reasoning</code> broken post-Daisy++	Remove hook; was never officially supported (Daisy bug)	Section 6b
Agent hits 10-iteration limit	Redesign flow to stay under limit; no config override today	Section 6b
Topic selector responding directly (not routing)	Add explicit "NEVER respond from this topic, ALWAYS route" instruction to topic selector	Section 4
Agent looping with processing limit	Known open bug (Case #473584957); simplify flow and reduce iteration depth	Section 6b

Resources & Tools

Debugging:

- Session Tracing (STDM): Setup > Session Tracing
- Agent Analytics: Setup > Agent Analytics
- Debug Logs: Setup > Debug Logs > Enable for Agent User

Documentation:

- Maintain CLAUDE.md for project context
- Document all variables in spreadsheet
- Keep topology diagram of topic relationships

Testing:

- Test in sandbox before production
- Use test variables during development
- Create regression test suite (20+ test cases)

Monitoring:

- Set up Agent Analytics dashboard
 - Weekly review of key metrics
 - Alert on escalation rate > 25%
-

Conclusion

The maturation of Agentforce from pure prompt-based agents to hybrid deterministic+agentic architectures represents the key to enterprise success. The most successful implementations:

1. **Start simple:** 3-5 core topics, proven patterns
2. **Add determinism:** Critical paths use flows/Apex
3. **Optimize progressively:** Based on real usage data
4. **Balance appropriately:** AI where valuable, code where reliable
5. **Monitor continuously:** Analytics-driven improvements

The Golden Rule: If you can't accept the agent being wrong 5-10% of the time, don't use an LLM for that decision - use code.

Document compiled from 10 months of Slack conversations, customer implementations, and field learnings (June 2025 – June 2026). Last updated: June 25, 2026. Sources: [#technical-support-new-agent-builder-and-script](#) and [adlc-customer-adoption](#).